

cA2A: Confidential Agent-to-Agent Delegation as a Profile on A2A

Rishabh Poddar Aaron Fulkerson Imran Siddique
OPAQUE Systems

<https://github.com/agenttrust-io/ca2a>

October 1, 2026 Technical report, version 2

Abstract

The Agent2Agent (A2A) protocol moves tasks between agents, and its Signed Agent Card lets a client check that a domain owner issued a card. The card does not bound the authority a delegating agent passes on, establish what code a peer runs, keep a task payload from the peer’s host, or leave an offline record of who delegated what to whom. This technical report describes cA2A (Confidential A2A), a trust profile layered on A2A rather than a new transport. It composes four mechanisms: signed delegation credentials whose scope can only narrow at each hop, appraisal of a peer’s attestation evidence before a task is sent, a payload sealed to the channel key that evidence vouches for, and a signed per-hop provenance record linked to its parent. Attenuated delegation and provenance binding are covered by prior capability-token work and IETF drafts; the contribution here is their composition on A2A with an open implementation. We state six properties and report software experiments rerun against ca2a 0.3.1: attenuation checks over 5,400 generated chains, rejection of in-chain replay and cross-chain splicing, intersection of delegated scope with local policy, sealed-payload behavior at the cryptographic layer, structural checks on linked provenance records, and a cross-operator attestation protocol exercised with synthetic evidence. Chain verification cost about 0.22 ms per hop in this environment. These results do not show that a peer’s key is confined to attested code or that attestation works across independent operators. Recorded hardware runs cover one-directional appraisal of an Intel TDX peer by an AMD SEV-SNP peer in another cloud and a same-operator mutual SEV-SNP diagnostic; mutual attestation between independent operators has not been demonstrated.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. Version 2 corrects one design claim that version 1 (September 27, 2026) overstated: the chain checks do not bind a delegation chain to a task or execution, so a delegate can reuse a valid chain across tasks. It adds three related citations. Results are unchanged. Version 1 revised a draft dated July 2026. Every software experiment was rerun on September 27, 2026 against ca2a 0.3.1 (commit 7d90823), the current release; the July results came from an unpinned earlier checkout and are retained beside the rerun. Correctness counts are unchanged. Timings changed: chain verification rose from about 195 to about 216 microseconds per hop, and sealed-channel throughput at 256 KiB fell from 266 to 212 MiB/s. Text, tables, and figures report the rerun. Status statements for C3 to C6 were revised to match the 0.3.1 code, security fixes made after the original evaluation are reported in Section 6, and the related work was corrected and extended. No hardware run was repeated. Current behavior, requirements, and limitations are maintained in the project repository.

Patent Pending.

Contents

1	Introduction	3
2	Background	4
2.1	A2A and the Signed Agent Card	4

2.2	MCP and cMCP as the Tool-Side Analog	4
2.3	Capability Attenuation and Macaroons	5
2.4	Trusted Execution Environments and Remote Attestation	5
2.5	Cedar Policy Language	5
2.6	Provenance and Transparency	6
2.7	TRACE as the Record Format	6
3	Design	6
3.1	Profile, not protocol	6
3.2	Where cA2A attaches to A2A	6
3.3	Threat model	7
3.4	Primitive 1: attenuated delegation	7
3.5	Primitive 2: runtime peer attestation	8
3.6	Primitive 3: sealed peer channel	8
3.7	Primitive 4: provenance DAG	9
3.8	Scope-policy intersection	10
3.9	Primitives to properties	10
4	Security Properties	10
5	Evaluation	12
5.1	Environment and reproducibility	13
5.2	C1: Attenuation Soundness	14
5.3	Verification Latency vs Chain Depth	14
5.4	C2: In-Chain Replay and Splice Rejection	14
5.5	C3: Scope-Policy Intersection	15
5.6	C4: Sealed-Payload Confidentiality	15
5.7	C5: Provenance Integrity	16
5.8	C6: Cross-Operator Attestation	16
6	Findings after the evaluation	17
7	Related Work	18
8	Conclusion	19

1 Introduction

The Agent2Agent (A2A) protocol [A2A Project, 2026] defines how one agent discovers another and hands it a task. It was contributed to the Linux Foundation, and on August 17, 2026 it joined the Agentic AI Foundation as a hosted project, which reported backing from more than 150 organizations [Agentic AI Foundation, 2026]. Teams that need agents from different vendors to exchange work increasingly have a common transport to build on.

A2A’s trust anchor is the Signed Agent Card, which lets a client check that the owner of a domain issued the card. It does not establish that a peer runs attested, unmodified code; that a delegating agent actually holds the authority it passes on; that a task payload is confidential to the intended peer; or that there is an auditable record of who delegated what to whom and under which policy.

This gap matters once one agent delegates to another. Agent A hands a task to B, and B hands part of it to C. Nothing in the transport constrains B or C to the authority A actually held. A card-verified peer can request more than it was granted, and the request will look valid at every hop. This is the confused deputy problem [Hardy, 1988], now across trust domains that never negotiated a shared authority model. Recent work treats verifiable delegation for agents as open: Prakash reports finding no prior implemented protocol that jointly combined verifiable delegation, holder-side attenuation, chained policy, transport bindings, and provenance records, and proposes one [Prakash, 2026]; Forough et al. list compound attestation for multi-hop agent chains among the open challenges for confidential agentic AI [Forough et al., 2026]. Mutual TLS and OAuth scopes secure the connection and assert an identity; they do not attenuate authority per hop, appraise runtime integrity, or seal a payload to an appraised peer.

This report describes cA2A (Confidential A2A), a trust profile layered on A2A rather than a competing transport. cA2A does not move tasks between agents; A2A does that. It defines the trust envelope around a delegated task, in the way TRACE [Poddar et al., 2026b] binds to RATS [Birkholz et al., 2023] and EAT [Lundblade et al., 2025] rather than replacing them. cA2A composes four mechanisms:

- Attenuated delegation. Each hop carries a signed delegation credential whose scope is a subset of its parent’s. Depth is bounded, the root issuer must be trusted by the verifier, and a credential identifier cannot repeat within a chain.
- Runtime attestation. A caller appraises a peer’s attestation evidence, which binds the peer’s channel key to a measurement, before it sends a delegated task.
- Sealed peer channel. The task payload is sealed to the channel key the appraised evidence vouches for, so only the holder of the matching private key can open it.
- Provenance record. Each hop emits a signed TRACE record referencing its parent record’s hash and the delegation credential it acted under, so an auditor can check the path offline.

None of these mechanisms is new in isolation. Macaroons introduced offline attenuation of bearer credentials [Birgisson et al., 2014], and recent agent-specific work specifies per-hop attenuation with Cedar policies and parent-swap resistance [Rampalli, 2026b] or implements attenuated, provenance-bound capability tokens across MCP and A2A [Prakash, 2026]. Section 7 compares them.

Contributions

- The cA2A profile. A binding on A2A, and on the IETF RATS, EAT, and SCITT [Birkholz et al., 2026] work, that composes attenuated delegation, attestation-gated sealing, and signed provenance for agent-to-agent calls without changing the A2A transport.
- An open reference implementation. The delegation credential model and offline chain verifier, the inbound decision path, the sealed channel, attestation verifiers for AMD SEV-SNP, Intel TDX, and TPM 2.0 evidence, signed TRACE provenance, and a CLI, released under the MIT license as ca2a-runtime [cA2A Contributors, 2026]. Delegation semantics mirror those of agent-manifest [Siddique, 2026], and hardware-evidence parsing and certificate-chain checks are shared with it.

- Six properties with their evidence stated. We state six properties (Section 4) and report software experiments rerun against ca2a 0.3.1 (Section 5). C1 (attenuation soundness) and C2 (in-chain replay and splice rejection) are validated in software. C3 (scope-policy intersection) is validated in software, including the inbound path of the reference transport. C4 (sealed-payload confidentiality) is validated at the cryptographic layer only. C5 (provenance integrity) is validated for structural consistency by the experiment, while authenticity rests on signed records covered by the project’s tests. C6 (cross-operator attestation) is exercised with synthetic evidence and is not validated on hardware across independent operators.

Paper structure

Section 2 covers A2A and its Signed Agent Card, TEE attestation, the TRACE record format, Cedar policy, and provenance. Section 3 describes the cA2A profile and how its mechanisms compose. Section 4 states the six properties and the status of each. Section 5 presents the rerun experiments. Section 6 reports findings made after the original evaluation. Section 7 surveys related work, and Section 8 concludes.

2 Background

This section grounds cA2A in the protocols and mechanisms it composes. We describe A2A and its Signed Agent Card, the tool-side analog in MCP and cMCP, capability attenuation and how agent-manifest realizes a signed delegation chain, trusted execution environments and remote attestation, the Cedar policy language, and the provenance and transparency work that the TRACE record format draws on. None of this section is a novel claim; it fixes terminology and states what each cited system provides.

2.1 A2A and the Signed Agent Card

The Agent2Agent (A2A) protocol [A2A Project, 2026] is a protocol for agent-to-agent communication. An agent publishes an Agent Card, a JSON document that advertises the agent’s identity, endpoint, and the skills it offers. A client agent reads a peer’s card, then sends it a task, a unit of work the peer executes and reports back on. A2A defines how tasks are moved and how their state transitions; it does not define an authorization model for what a peer is permitted to do with a task once it holds it.

The Signed Agent Card is A2A’s trust anchor. The card carries a signature that lets a client verify that the claimed domain owner issued it. This answers one question: did this domain publish this card. It does not establish that the peer is running attested code, that a delegating agent holds the authority it forwards, that the task payload is confidential to the peer, or that there is an unbroken record of who delegated what to whom. cA2A adds these guarantees as a profile on A2A rather than as a competing transport.

2.2 MCP and cMCP as the Tool-Side Analog

The Model Context Protocol (MCP) [Model Context Protocol Project, 2026] standardizes how an agent discovers and invokes tools hosted on external servers. Where A2A governs agent-to-agent calls, MCP governs the agent-to-tool call. MCP, like A2A, defines discovery and invocation but not authorization: it specifies no policy model, requires no audit record, and gives a client no way to confirm that a cached tool definition is still the definition the server will execute.

cMCP [Poddar et al., 2026a] closes those gaps for the tool call. It interposes a gateway that evaluates a Cedar policy per call, maintains a tamper-evident audit chain, and emits a signed TRACE Claim that any verifier can check offline. cA2A is the agent-to-agent counterpart of the same program: it takes the tool-side pattern of attested, policy-bound, provable calls and applies it to the delegation edge between agents. cA2A reuses cMCP’s TEE provider pattern and runs the same Cedar engine (`cedarpy`) for local policy.

2.3 Capability Attenuation and Macaroons

The confused deputy problem [Hardy, 1988] arises when a component with authority is induced to exercise it on behalf of a caller who should not have that authority. In a multi-agent workflow the deputy is any agent that accepts a narrow task and then acts with broader authority than the task granted. The capability discipline answers this by making authority a value that is passed explicitly and can only be narrowed as it is passed on.

Macaroons [Birgisson et al., 2014] are the reference construction for attenuable bearer credentials. A macaroon is a token whose holder can append caveats that restrict its use, producing a strictly less powerful token without contacting the issuer; the chain of caveats is bound by a keyed-HMAC construction so that no caveat can be dropped. The essential property for delegation is monotone attenuation: a derived credential can only be a subset of the one it derives from.

agent-manifest [Siddique, 2026] implements this discipline as a signed A2A delegation chain. Each hop is a `DelegationCredential` whose signed body names the issuer, the subject, the granted scope, a depth, and the parent hop’s identifier. The body is signed with Ed25519 [Josefsson and Liusvaara, 2017] over a canonical JSON encoding aligned with RFC 8785 [Rundgren et al., 2020], so the exact bytes that were signed are reconstructable by any verifier. Verification enforces signature validity at every hop, continuity of the issuer-to-subject and parent-to-child links, a bounded depth, non-repetition of credential identifiers to defeat replay, and the subset check on scope that enforces attenuation. cA2A implements the same semantics in its own verifier, which the runtime calls on every inbound peer request, and signs over RFC 8785 canonical bytes so that credentials are cross-verifiable with agent-manifest. The attenuation subset check is what forecloses the confused-deputy escalation.

2.4 Trusted Execution Environments and Remote Attestation

A Trusted Execution Environment (TEE) is a CPU-enforced isolation boundary that provides two properties cA2A depends on: confidentiality, so that memory inside the TEE is inaccessible to software outside it including the hypervisor and host OS, and remote attestation, so that the TEE can produce a hardware-signed measurement report proving to a remote verifier which code is running inside. AMD SEV-SNP [Advanced Micro Devices, Inc., 2020] and Intel TDX [Intel Corporation, 2026] are the two mainstream VM-level implementations; a survey of confidential machine learning provides broader context [Mo et al., 2024].

The measurement report is the primitive of interest. Both platforms let the enclave place application-chosen bytes into a report field bound to the hardware measurement, which is how a public key can be tied to a specific attested measurement. The IETF RATS architecture [Birkholz et al., 2023] gives the vendor-neutral roles for this exchange, an attester producing evidence and a verifier appraising it, and the Entity Attestation Token (EAT) [Lundblade et al., 2025] gives a portable token format for carrying attestation claims. cA2A binds a peer’s channel key to its attested measurement through this mechanism, so that a caller can seal a payload to a key the appraised evidence vouches for. Whether that key is in fact confined to the measured code depends on the platform and on what the measurement covers; Sections 4 and 6 state what has and has not been shown.

2.5 Cedar Policy Language

Cedar [Cutler et al., 2024] is a policy language and evaluation engine that expresses access control as `permit` and `forbid` rules over a principal, action, and resource, with optional conditions over a context object. Cedar’s structural typing keeps evaluation decidable in bounded time, which matters when a policy check sits on the critical path of accepting a delegated task. cA2A uses Cedar to model the intersection of a forwarded delegation scope with a peer’s local policy, so that a peer never acts beyond either the authority delegated to it or its own operator’s rules. Property 3 states this intersection and Section 5.5 evaluates it.

2.6 Provenance and Transparency

An offline-verifiable record of who delegated what to whom is a provenance problem. The W3C PROV data model [Moreau and Missier, 2013] gives the vocabulary for describing entities, activities, and the agents responsible for them, and the relationships that link a derived artifact to what it was derived from. cA2A’s per-hop records form a delegation DAG in exactly this shape: each record names its parent, so the set of records across a multi-hop workflow reconstructs the delegation tree.

Transparency logs make records non-repudiable and their omission detectable. Certificate Transparency [Laure et al., 2013] established the pattern of an append-only, cryptographically verifiable log for security-critical statements, and the IETF SCITT architecture [Birkholz et al., 2026] generalizes it to supply-chain claims, giving neutral roles for registering and auditing signed statements. cA2A follows the same profile-over-standard posture toward SCITT that it takes toward A2A: it binds to the transparency architecture rather than reinventing it.

2.7 TRACE as the Record Format

TRACE (Trust, Runtime Attestation, and Compliance Evidence) [Poddar et al., 2026b] is the open record format that cA2A emits per delegation hop. A TRACE record is a signed JSON document that carries a subject, a runtime measurement, and profile-specific context. cA2A defines an A2A profile of TRACE that adds a delegation-link block to each record: the hash of the parent hop’s record and the identifier of the delegation credential the hop acted under. A root hop omits the block; every subsequent hop references its parent. Because each record is signed and references its parent by hash, a verifier can rebuild the parent links, confirm each hop acted under a credential whose chain verifies, and confirm that each record was signed by the key the chain names as that hop’s delegate. These checks run offline, given the recipient’s trusted keys and the delegation chain. They establish what the signers declared, not that the declared work completed or that the submitted path is the complete history, and the current verifiers accept one ordered root-to-leaf path rather than an arbitrary branching graph. This is the record substrate for the provenance property in Section 4.

3 Design

3.1 Profile, not protocol

cA2A is a trust profile layered on the Agent2Agent (A2A) protocol [A2A Project, 2026], not a competing transport. A2A moves tasks between agents and, through the Signed Agent Card, answers exactly one question: did the domain owner issue this card. It does not establish that a peer runs attested code, that a delegating agent actually holds the authority it passes on, that a task payload is confidential to the intended peer, or that there is an unbroken record of who delegated what to whom. cA2A adds that trust envelope around a delegated task without redefining how the task is carried. It binds to A2A the way TRACE [Poddar et al., 2026b] binds to the IETF RATS [Birkholz et al., 2023], EAT [Lundblade et al., 2025], and SCITT [Birkholz et al., 2026] work: as an overlay, not a replacement. The profile is written to be independent of any one organization, cloud, or TEE vendor; the implementation currently appraises AMD SEV-SNP, Intel TDX, and TPM 2.0 evidence.

Four design tenets constrain every primitive below. First, profile, not protocol: reuse A2A and the RATS/EAT/SCITT stack rather than reinvent them. Second, neutral by construction: no coupling to a single silicon vendor, cloud, or model platform. Third, fail closed: absence of evidence is denial, not a warning; a missing attestation or an unverifiable chain denies the call. Fourth, offline verifiable: a third party can verify a delegation chain and its provenance DAG without contacting or trusting any operator.

3.2 Where cA2A attaches to A2A

cA2A interposes on the trust decision that A2A leaves undefined: the moment one agent receives a delegated task from a peer and must decide whether to accept it. On an inbound peer call, the caller appraises the

callee’s attestation offer and seals the payload to the channel key it vouches for. The callee then, in order, (i) verifies the delegation chain (trusted root, signature, validity window, continuity, attenuation, in-chain replay, and revocation when a snapshot is supplied), (ii) checks a holder proof showing that the caller holds the key the chain delegates to, (iii) intersects the delegated scope with local policy, (iv) appraises the caller’s own attestation when the callee requires it, (v) decides the requested capability, and (vi) only then opens the sealed payload and emits a linked provenance record for the allow or deny decision. The A2A transport, agent identity issuance beyond delegation, and model governance are all out of scope: cA2A governs only the delegation path between agents.

3.3 Threat model

cA2A defends the delegation path between agents against a capable adversary who may operate a peer agent in a trust domain cA2A does not control; present a valid A2A Signed Agent Card while running tampered or unmeasured code; sit on the network between two agents or operate the host a peer runs on; and attempt to widen a delegated grant, replay a credential into another chain, or reparent a provenance record. The assets and their defenses are summarized in Table 1.

Out of adversary scope is breaking the underlying cryptographic primitives (Ed25519 [Josefsson and Liusvaara, 2017], the hash function) or compromising TEE firmware or hardware microcode [Advanced Micro Devices, Inc., 2020, Intel Corporation, 2026]. Those are the trust anchors. The central attack cA2A addresses is the confused deputy [Hardy, 1988] at machine speed: a delegate that accepts a narrow task and then acts with authority it was never granted.

What ca2a 0.3.1 defends is narrower than this threat model. Attenuated delegation, scope-policy intersection on the inbound path, the sealed channel’s cryptography, and signed provenance ship and are tested in software (Section 5). Attestation verifiers for SEV-SNP, TDX, and TPM evidence ship, and the live path can require hardware assurance, but this is opt in: the reference client and server default to software assurance. Hardware runs so far are one-directional across two clouds or mutual within one operator (Section 3.5). Confidentiality against a hosting operator additionally depends on platform properties the recorded hosts lacked (Section 6). cA2A should therefore not be described as attested across independent trust domains, and task payloads should not be assumed protected from a hosting operator.

3.4 Primitive 1: attenuated delegation

A delegation chain is a root-to-leaf list of signed credentials that lets Agent A hand a bounded slice of its authority to B, and B a still-smaller slice to C, with each grant provably within the one above it. The semantics mirror the signed delegation chain implemented and tested in agent-manifest [Siddique, 2026], and follow the capability-attenuation model of macaroons [Birgisson et al., 2014]; cA2A implements that verifier and its runtime calls it on every inbound peer request.

Each hop carries a `DelegationCredential` with a signed body and a detached Ed25519 signature. The body has `credential_id` (a unique id for the hop), `issuer` and `subject` (the delegator’s and delegate’s public keys), `scope` (the set of capabilities granted at this hop), `depth` (0 at the root, +1 per hop), and `parent_id` (the parent hop’s `credential_id`, null at the root), with optional `not_before` and `not_after` bounds. The signed bytes are the RFC 8785 [Rundgren et al., 2020] canonical form of the body, with `scope` as a sorted array; floats are refused and integers are bounded to the range RFC 8785 can represent exactly.

The verifier, `verify_chain`, raises the specific error for the first invariant that fails. The root issuer must be in the verifier’s trusted set (else `UNTRUSTED_DELEGATION_ROOT`); every hop’s signature must verify against its issuer (else `INVALID_CREDENTIAL`) and its validity window must contain the evaluation time; the root must have no parent and depth 0, each hop’s `parent_id` must equal the previous hop’s `credential_id`, and each hop’s `issuer` must equal the previous hop’s `subject` (else `BROKEN_DELEGATION_LINK`); each hop’s depth must be the previous depth +1 and at most `max_depth` (else `DELEGATION_DEPTH_EXCEEDED`); each hop’s scope must be a subset of its parent’s scope (else `SCOPE_ESCALATION`); and no `credential_id` may repeat (else `CREDENTIAL_REPLAY`). Given a signed revocation snapshot, a hop revoked by its issuer or any issuer above it is refused (`CREDENTIAL_REVOKED`). The verifier keeps no global history of used credentials, so

replay of a whole valid chain by a different party is addressed by the holder proof in the inbound path rather than by `verify_chain`.

Consider the chain

$$A \xrightarrow{S_A} B \xrightarrow{S_B \subseteq S_A} C, \quad S_C \subseteq S_B \subseteq S_A.$$

A issues B a child credential whose scope S_B is a subset of A’s grant S_A , signed over the canonical body. B in turn issues C a credential with $S_C \subseteq S_B$. The subset check at every hop is the confused-deputy defense within a chain: it forecloses B accepting a narrow task from A and then acting with authority A never granted. Attenuation is the whole point. The `parent_id`, `issuer/subject` continuity, and unique-id checks close the complementary attack of replaying a credential within a chain or splicing one from another chain into it. They do not bind a chain to a particular task or execution: a credential carries no task or execution identifier, so the delegate can present the same valid chain with a different request. Making execution lineage part of the authorization decision is the subject of history-based access control [Abadi and Fournet, 2003], provenance-based access control [Park et al., 2012], and, for agent delegation, the PIC model [Gallo, 2026]. cA2A does not provide it; authenticated lineage records (Section 6) say which hop acted under which credential, which is not the same as refusing authority outside its lineage. This primitive is implemented and validated (Properties 1 and 2).

3.5 Primitive 2: runtime peer attestation

Before a peer is trusted with a delegated task, it presents evidence of the code it runs. cA2A follows the TEE provider pattern of cMCP [Poddar et al., 2026a]. A provider implements `detect()`, returning whether it is available on the current host, and `attest(public_key, nonce)`, returning an `AttestationReport` that binds a public key to the host’s hardware measurement under a nonce. The report carries `platform`, `measurement`, the bound `public_key`, and the `nonce`, following the RATS evidence model [Birkholz et al., 2023] and EAT claims format [Lundblade et al., 2025].

The providers are a `software-only` backend (for development and CI, which reports `platform: software-only` and never a hardware platform string) and three hardware backends: `tpm` (TPM 2.0 or vTPM), `sev-snp` (AMD SEV-SNP [Advanced Micro Devices, Inc., 2020]), and `tdx` (Intel TDX [Intel Corporation, 2026]).

Implementation status at 0.3.1. The SEV-SNP and TDX verifiers check certificate chains to pinned AMD and Intel roots, report signatures, and key binding, and the project records runs against genuine evidence from both platforms; for this edition the TDX verifier was re-run against a public GCP TDX quote (Section 5). The TPM verifier checks quote structure, binding, and attestation-key signature; its certificate chain depends on a caller-supplied vendor root, and on one Azure host generation no chain to a pinnable root could be assembled. SEV-SNP and TDX collectors use the kernel configs-TSM interface and have run on GCP confidential VMs; an Azure SEV-SNP guest is paravisor-mediated, cannot set `REPORT_DATA` directly, and roots its channel key through the vTPM instead. The project records two live runs. On July 27, 2026 an Azure SEV-SNP caller appraised a GCP TDX peer’s quote against the Intel root, sealed a delegated task to the bound key, and the peer allowed an in-scope capability and refused an out-of-scope one; the callee did not appraise the caller, and one operator ran both sides. On September 17, 2026 two GCP SEV-SNP guests in one project appraised each other and exchanged sealed non-secret payloads in both directions. Neither run is mutual attestation between independent operators, so Property 6 is not validated on hardware. Verification fails closed when required evidence is absent.

3.6 Primitive 3: sealed peer channel

When A sends B a task payload, that payload crosses a network and lands in B’s memory. If B is in another trust domain, mTLS protects the pipe but not the endpoint: the operator hosting B, or a connectivity provider between them, can read plaintext. The sealed channel encrypts the payload to B’s channel public key, and B’s attestation evidence binds that key to a measurement, extending the attestation-gated key pattern cMCP uses for the agent-to-gateway case to the peer-to-peer case. The connectivity path sees

ciphertext. The interface is `SealedChannel(peer_public_key).seal` on the sender and `open_sealed` with the private key on the receiver.

Cryptographic layer: implemented and validated. The channel ships over X25519 ECDH, HKDF-SHA256 key derivation, and ChaCha20-Poly1305 AEAD. A payload sealed to a peer’s channel key decrypts only with that key, the connectivity path sees only ciphertext, and a tampered blob fails AEAD authentication so `open` fails closed rather than returning plaintext. Property 4 validates exactly this at the cryptographic layer (Section 5). Binding to attestation. On the live path the caller seals only to a channel key that its verifier has appraised (`ca2a_runtime.attestation`). In software mode that appraisal has assurance `none`; with a hardware verifier configured and `require_hardware` set, a software offer is refused before sealing. Even with hardware appraisal, the channel key is an ordinary key held by a process in the guest. In the September 17 diagnostic the launch measurements did not cover application code installed after boot, and the SEV-SNP hosts reported that ciphertext hiding was not enabled. Confidentiality against the hosting operator therefore remains the dependency that Property 4 names.

Figure 1 shows the seal-and-open path. The sender derives a one-time symmetric key from an ephemeral X25519 ECDH to the peer’s channel public key, runs it through HKDF-SHA256, and encrypts the payload under ChaCha20-Poly1305. The connectivity path sees only the sealed blob; only the holder of the peer’s channel private key recovers the plaintext.

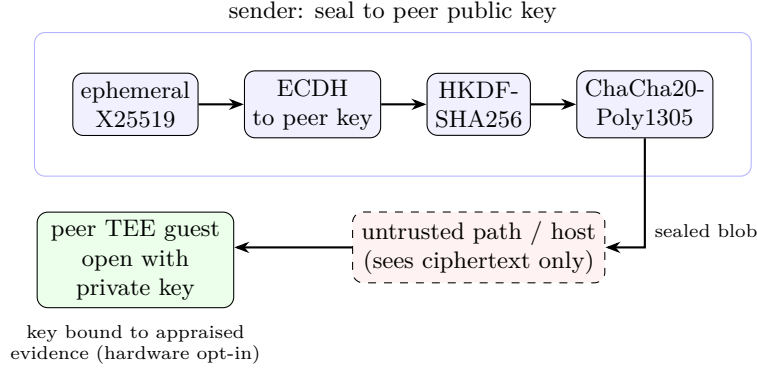


Figure 1: Sealed peer channel. An ephemeral X25519 ECDH to the peer’s channel key is run through HKDF-SHA256 to derive a symmetric key, and the payload is sealed under ChaCha20-Poly1305 AEAD. The untrusted connectivity path sees only ciphertext; only the private key matching the appraised channel key opens the blob. Whether the peer’s host can reach that key depends on the platform (Section 6).

3.7 Primitive 4: provenance DAG

ca2A emits a signed TRACE record per delegation hop [Poddar et al., 2026b]. The A2A profile adds a delegation-link block to the base TRACE record, which otherwise carries a subject but no notion of a parent. The block has `delegation.parent_record_hash`, the hash of the parent hop’s TRACE record, and `delegation.credential_id`, the id of the delegation credential this hop acted under. A root hop omits the block; each subsequent hop references its parent, so the set of records across A to B to C reconstructs the delegation tree. The construction follows the W3C PROV data model [Moreau and Missier, 2013] for lineage and the SCITT [Birkholz et al., 2026] approach to transparency over signed statements.

Given the records for a workflow and the recipient’s trusted signing keys, a verifier (`verify_trace_dag`) checks each record’s signature and rebuilds the parent links from `parent_record_hash`; a cross-check (`cross_check_trace_dag`) then aligns the path with a separately verified delegation chain, requiring matching path length and credential identifiers and that each record was signed by the key the chain names as that hop’s delegate. Tampering any signed record breaks its signature, and altering a parent link breaks the chain of hashes. The runtime also keeps an unsigned `DelegationRecord` form whose hash links give only structural consistency: a party able to rewrite the whole path can recompute every link, so since ca2a 0.3.0 the CLI reports such a path as structurally consistent but not authenticated. Property 5 and Section 5.7

distinguish the two. Both verifiers accept one ordered root-to-leaf path. The A2A profile’s delegation-link block is defined in trace-spec.

Figure 2 shows both halves together: the attenuated delegation chain on top, where each hop’s scope is a strict subset of its parent’s, and the hash-linked provenance path below, where each record carries the hash of its parent record and names the credential the hop acted under. The two are bound: record i must match credential i and, for signed records, must be signed by that credential’s delegate key.

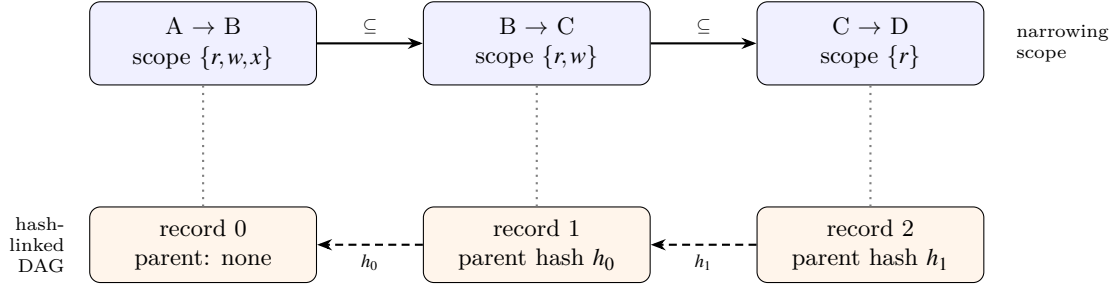


Figure 2: Attenuated delegation chain (top, solid) and its hash-linked provenance DAG (bottom, dashed). Each delegation hop’s scope is a provable subset ($\subseteq$) of its parent’s; each provenance record stores the SHA-256 hash of its parent record. The dotted lines bind record i to credential i . With signed records and trusted keys, tampering, reparenting, or attributing a record to the wrong delegate is detected offline; hash links alone give only structural consistency.

3.8 Scope-policy intersection

Verifying that a delegated grant is well-formed is not the same as deciding that the local peer is willing to act on it. On an inbound call, the cA2A runtime is specified to intersect the delegated scope S_B with the peer’s local authorization policy, expressed in Cedar [Cutler et al., 2024] or as a simple allow set, and to act only on the intersection $S_B \cap P_{\text{local}}$. A grant is thus bounded from above by both the delegator’s authority (attenuation) and the delegate’s own policy, so neither a broad grant nor a permissive local default alone can widen what B actually does.

Implementation status. The intersection runs against either a Cedar engine (`cedarpy` 4.8.7) or the set-based `LocalPolicy`, inside the inbound decision path (`handle_peer_request`). That path is served by the reference HTTP transport (since ca2a commit 7d33445, July 2026) and by a bridge to the official `a2a-sdk` (commit f26e2bc), which the project tests over a local JSON-RPC loopback with `a2a-sdk` 1.1.4. Property 3 is evaluated in Section 5.5.

3.9 Primitives to properties

Table 1 maps each primitive to the asset it protects, the security property it supports, and its status at ca2a 0.3.1. The software results need no hardware TEE and can be rerun from the open-source codebase; the hardware dependencies are stated per row.

4 Security Properties

We state six security properties of the cA2A profile. Each is an informal invariant over the runtime; formal proofs are left to future work. Each carries a status line for ca2a 0.3.1 that names the layer at which it has been checked and what that check does not cover. This section states the guarantees and mechanisms; results appear in Section 5.

Property 1 (Attenuation Soundness).

Table 1: cA2A primitives, the assets they protect, the properties they support, and status. “Validated” means backed by the rerun software experiments of Section 5, at the layer stated.

Primitive	Asset protected	Property	Status
Attenuated delegation	Bounded authority (no delegate exceeds its grant)	C1 attenuation soundness; C2 in-chain replay and splice rejection	Validated (software)
Scope-policy intersection	Bounded authority under local policy	C3 scope-policy intersection	Validated (software), including the reference inbound path
Runtime peer attestation	Peer integrity (only appraised code trusted)	C6 cross-operator attestation	Synthetic evidence only; hardware runs one-directional or same-operator
Sealed peer channel	Task confidentiality to the intended peer	C4 sealed-payload confidentiality	Validated (cryptographic layer); key confinement not shown
Provenance record	Provenance (signed record of who delegated what)	C5 provenance integrity	Structural checks validated; signed path covered by project tests

Status: validated in software. For any delegation chain of signed credentials rooted at a trusted issuer, the scope granted at each hop is a subset of the scope of its parent, and any attempt to grant a capability the parent does not hold is rejected. This is the confused-deputy defense [Hardy, 1988] within a chain: without it, a delegate could accept a narrow task and then act with authority the delegator never conferred. The mechanism is a per-hop subset check in `verify_chain`: each credential carries a `scope` set, a `depth`, a `parent_id`, and a detached Ed25519 signature [Josefsson and Liusvaara, 2017] over the canonical body. A hop whose `scope` is not contained in its parent’s `scope` raises `ScopeEscalation`; a monotone-narrowing chain verifies. The semantics mirror agent-manifest [Siddique, 2026] and follow the capability-attenuation model of macaroons [Birgisson et al., 2014].

Property 2 (In-Chain Replay and Splice Rejection).

Status: validated in software. A credential presented twice within a chain, or lifted from one chain and spliced into another, is rejected. This forecloses splicing a legitimately issued grant into an unrelated delegation to inherit authority out of context. The mechanism combines two checks in `verify_chain`. First, each hop’s `credential_id` must be unique within the chain; a repeat raises `CredentialReplay`. Second, structural continuity binds each hop to the previous one: a hop’s `parent_id` must equal the previous hop’s `credential_id` and its `issuer` must equal the previous hop’s `subject`. A credential spliced from a foreign chain breaks one of these links and raises `BrokenDelegationLink`. The property does not cover presentation of a whole valid chain by a party other than its delegate, since the verifier keeps no global history; on the live path that case is refused by the holder proof (Section 6). Nor does it cover the delegate itself presenting a valid chain with a request from a different task, since nothing binds a chain to an execution (Section 3).

Property 3 (Scope-Policy Intersection).

Status: validated in software, including the inbound path of the reference transport. The effective permission at an inbound peer call is the delegated scope intersected with the receiving agent’s local policy [Cutler et al., 2024]: a capability is exercised only if the upstream chain granted it and the local policy independently permits it. This makes each domain the final authority over what runs on its own resources, so an over-broad delegated grant cannot force action the receiver’s policy forbids. The mechanism is the intersection of the verified chain scope (Property 1) with the local policy decision, evaluated in `effective_scope` and

`enforce_peer_call` in `ca2a_runtime.peer`; a capability outside the intersection raises `ScopeNotPermitted`. Local policy is either a Cedar policy set evaluated by `cedarpy` or a set-based `LocalPolicy`. The same decision runs inside `handle_peer_request`, which the reference HTTP transport and the `a2a-sdk` bridge call on each inbound request. The check does not cover third-party A2A servers or a production deployment.

Property 4 (Sealed-Payload Confidentiality).

Status: validated at the cryptographic layer. A task payload sent across a trust boundary decrypts only with the private key matching the channel key the caller appraised, so no party on the connectivity path can read plaintext. The mechanism seals the payload to the peer’s channel public key with an HPKE-style X25519, HKDF-SHA256, and ChaCha20-Poly1305 construction (`ca2a_runtime.channel`); only the peer’s private key opens it, an unrelated key fails, and a tampered ciphertext fails closed under AEAD authentication. What software cannot establish is that the private key is confined to the attested code, so that the operator hosting the peer cannot read it. That depends on hardware appraisal being required on the call, on the measurement covering the code that holds the key, and on platform protections such as SEV-SNP ciphertext hiding. Section 6 records that the measured hosts did not meet the last two conditions. Until they are met, confidential payloads should not be sent across a trust boundary in the belief that a compromised host cannot read them.

Property 5 (Provenance Integrity).

Status: structural checks validated in software; authenticated lineage covered by the project’s tests. The per-hop records for a workflow form a linked path that is bound to the delegation authority. With signed TRACE records and trusted signing keys, tampering with any record, reparenting a record, or attributing a record to a hop that did not sign it is detected offline [Moreau and Missier, 2013]. The mechanism adds a delegation-link block to each record: `parent_record_hash` links a hop to its parent, and `credential_id` names the credential the hop acted under. Altering a record changes its hash and breaks the child’s `parent_record_hash` link, raising `ProvenanceLinkBroken`. A cross-check binds the provenance path to the delegation chain of Property 1: record i must correspond to credential i , and a signed record must carry the delegate key that credential names. Hash links without signatures detect only partial edits: a writer who rewrites the whole path can recompute every link. The property does not establish that declared work completed or that the submitted path is complete.

Property 6 (Cross-Operator Attestation).

Status: exercised with synthetic evidence; not validated on hardware across independent operators. Two agents in independent trust domains, holding independently generated keys, each appraise the other’s runtime attestation before a delegated task is accepted, and a peer that loads a different binary than the one expected is detected. The mechanism follows the IETF RATS architecture [Birkholz et al., 2023]: each peer produces a hardware-signed attestation report binding its channel key and a nonce, and the counterpart verifies the report’s certificate chain and signature and compares its measurement with an expected value, so a binary swap changes the measurement and is rejected. The SEV-SNP, TDX, and TPM verifiers that this requires ship in ca2a 0.3.1 (Section 3.5). The recorded hardware runs are one-directional across two clouds, or mutual within one operator’s project; neither is mutual attestation between independent operators.

5 Evaluation

We evaluate the six properties of Section 4 against ca2a 0.3.1, the current release (commit 7d90823, September 26, 2026). Every experiment in this section was rerun on September 27, 2026. The original July 2026 results came from an earlier, unpinned checkout; where numbers changed, both are reported.

5.1 Environment and reproducibility

The experiments ran under CPython 3.12.10 (an x86-64 build running under emulation on a Windows 11 laptop with an ARM64 Snapdragon X1E80100 processor), with dependencies from the project’s runtime lock at that commit: `cryptography` 50.0.1, `cedarpy` 4.8.7, `agent-manifest` 0.12.0, `agenttrust-trace` 0.10.0, and `rfc8785` 0.1.4. The one deviation from the lock is `cbor2` 5.9.0 in place of 6.1.4, because the host’s application-control policy blocks the 6.x native extension. Timings are single runs under emulation and vary from run to run; they show how cost scales, not what a server would measure. No hardware TEE is needed. The rerun record, with every command’s output, is committed with the paper as `experiments/results/rerun-2026-09-27.txt`.

Two sets of experiments are used. The repository’s own per-claim experiments live under `experiments/` in `ca2a` [cA2A Contributors, 2026]; the three scale and cost experiments below ship with this report and import the runtime from a `ca2a` checkout named by `CA2A_ROOT`:

Listing 1: Rerunning the experiments

```
1 # in a ca2a checkout at tag v0.3.1
2 pip install -e ".[dev]"
3 python experiments/claim1-attenuation-soundness/run.py # C1 .. claim6 likewise
4 # in this report's directory
5 CA2A_ROOT=/path/to/ca2a python experiments/c1_attenuation_scale.py
6 CA2A_ROOT=/path/to/ca2a python experiments/verify_latency_vs_depth.py
7 CA2A_ROOT=/path/to/ca2a python experiments/sealed_channel_throughput.py
```

At 0.3.1, `verify_chain` refuses a chain whose root issuer the caller has not named as trusted (ca2a commit 23c980a), so the three report scripts now pass each chain’s root as the trusted set; no other logic changed. The project’s test suite at the same commit produced 779 passes and 11 skips: nine skips need a git checkout rather than the source archive used here, and two need hardware captures. Setting `CA2A_TDX_QUOTE` to a genuine GCP Intel TDX quote published in the `agent-manifest` repository ran one of those two: the TDX verifier appraised it to the pinned Intel SGX Root CA, and all 17 TDX tests passed. That is offline appraisal of one captured quote, not a live attested call. Table 2 summarizes the outcomes.

Table 2: Outcomes at `ca2a` 0.3.1, rerun September 27, 2026. The Status column names the layer at which each property was checked.

Property	Status	Result
C1 Attenuation soundness	Validated (software)	200/200 narrowing chains accepted and 200/200 escalations rejected with <code>ScopeEscalation</code> ; across depths 2 to 8, 2500/2500 accepted and 2500/2500 rejected
C2 In-chain replay and splice	Validated (software)	Duplicate <code>credential_id</code> raises <code>CredentialReplay</code> ; cross-chain splice raises <code>BrokenDelegationLink</code> ; 2/2 controls valid
C3 Scope-policy intersection	Validated (software)	Effective scope is delegated $\cap$ local policy; 1/1 allowed, 3/3 denied; the inbound transport tests exercise the same decision
C4 Sealed-payload confidentiality	Cryptographic layer	4/4: only the peer’s key opens the payload; the path sees ciphertext; wrong key and tamper fail closed. Key confinement to attested code not shown
C5 Provenance integrity	Structural (experiment)	Tampering one record changed 119 and 138 of 256 hash bits in two runs and raised <code>ProvenanceLinkBroken</code> ; reparenting detected; record i bound to credential i
C6 Cross-operator attestation	Synthetic evidence	4/4 with synthetic SEV-SNP vectors: independent keys, mutual appraisal, sealed delegation, binary swap rejected. No hardware run across independent operators

5.2 C1: Attenuation Soundness

Setup. Property 1 states that a child grant can never exceed its parent. The repository experiment generates 200 independent four-hop delegation chains. In the narrowing case, each hop drops exactly one capability from a six-capability root, so every hop is a strict subset of the one above it. In the escalating case, the third hop adds a fresh capability that no ancestor held. Each chain is signed hop by hop, so issuer continuity, depth, and `parent_id` linkage hold; only the scope relationship differs.

Results. All 200 narrowing chains are accepted, and all 200 escalating chains are rejected with `ScopeEscalation`; none is rejected by an unrelated error. A correctly signed credential that widens its scope is still rejected, so attenuation is enforced independently of signature validity.

Across chain depth. For each depth in $\{2, 3, 4, 6, 8\}$ we generated 500 narrowing chains and 500 escalation chains (each escalation injects a never-held capability at an interior hop) over a 16-capability root universe, 2500 of each in total. Every narrowing chain was accepted and every escalation was rejected with `ScopeEscalation`, with zero rejections by an unrelated error, matching the July result. The per-depth counts are in `experiments/results/c1_attenuation_scale.json`.

5.3 Verification Latency vs Chain Depth

Setup. Chain verification is the per-call cost a callee pays before it acts: trusted root, signature, validity, continuity, attenuation, and in-chain replay over every hop. We built one correctly signed chain at each depth in $\{1, 2, 4, 8, 16, 32, 64\}$, ran 200 warm-up iterations, then timed `verify_chain` over 2000 iterations per depth, reporting mean, p50, and p99. Chains are signed once up front, so the measurement excludes key generation.

Results. Cost grows linearly with depth, at about 216 microseconds per hop between depth 1 and depth 64 (July: about 195). A single hop verifies in 0.25 ms mean; a 64-hop chain verifies in 13.87 ms mean with p99 26.08 ms (July: 0.14 ms, 12.45 ms, and 22.15 ms). Table 3 lists both runs, and Figure 3 plots the rerun. The two runs differ in code version (the verifier gained trusted-root and validity-window checks after July) and in environment, so the difference is not attributed to either alone. At realistic depths of a few hops the cost stays near a millisecond or below. Raw values are in `experiments/results/verify_latency_vs_depth.json`.

Table 3: Original (July 1, 2026, unpinned checkout) and rerun (September 27, 2026, ca2a 0.3.1) timings. Both are single runs on developer hardware.

Measurement	July 2026	Rerun
<code>verify_chain</code> , depth 1, mean (ms)	0.14	0.25
<code>verify_chain</code> , depth 8, mean (ms)	1.73	2.07
<code>verify_chain</code> , depth 64, mean / p99 (ms)	12.45 / 22.15	13.87 / 26.08
Slope, depth 1 to 64 (μ s per hop)	195	216
Seal and open, 64 bytes, round trip (ms)	0.21	0.17
Seal and open, 256 KiB, throughput (MiB/s)	266	212

5.4 C2: In-Chain Replay and Splice Rejection

Setup. Property 2 states that a credential replayed within a chain or spliced from another chain is rejected. We build two independent, well-formed chains, A and B, each with its own key material and credential-id namespace. Three cases are checked: (1) a control in which both chains verify unmodified; (2) an intra-chain replay in which one hop’s `credential_id` is duplicated inside the chain; and (3) a cross-chain splice in which a validly signed credential minted for chain A is inserted into chain B.

Results. Both control chains verify (2/2). The duplicated `credential_id` raises `CredentialReplay`. The spliced credential is rejected with `BrokenDelegationLink` because its `parent_id` does not match chain B’s preceding credential. A valid signature is therefore necessary but not sufficient; continuity is enforced over

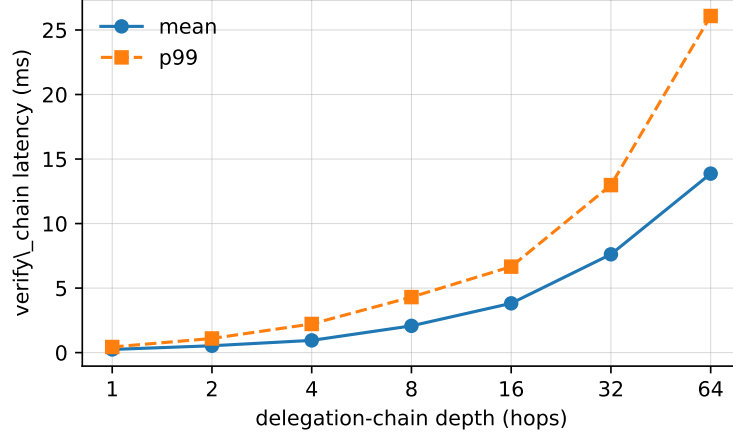


Figure 3: Chain-verification latency (mean and p99) versus delegation-chain depth, log-scale x-axis, 2000 timed iterations per depth, ca2a 0.3.1. Cost is linear in depth at about 216 microseconds per hop; a 64-hop chain verifies in 13.87 ms mean.

the chain, not per credential. Replay of a whole valid chain by a different party is outside this experiment; the project covers it with holder-proof tests.

5.5 C3: Scope-Policy Intersection

Setup. Property 3 states that the capability a caller may exercise against a peer is the intersection of its verified delegated scope with the callee’s local policy. We build a two-hop chain delegating a leaf scope of `{read, write}` (the root also held `admin`) and set the callee’s local policy to allow `{read, audit}`. We then request four capabilities through `effective_scope` and `enforce_peer_call`: `read` (delegated and allowed), `write` (delegated but not locally allowed), `audit` (locally allowed but never delegated), and `admin` (neither).

Results. The effective scope is exactly `{read}`. Only `read` is allowed (1/1); `write`, `audit`, and `admin` are each denied with `ScopeNotPermitted` (3/3). A capability is exercised only when the chain granted it and the local policy permits it. The experiment uses the set-based `LocalPolicy`; the project’s unit tests exercise the Cedar engine, and its live-call and `a2a-sdk` loopback tests, which passed in the rerun, drive the same decision through an inbound HTTP and JSON-RPC request on the local host, including a denial for an out-of-scope capability. None of these is a deployment across a real network boundary.

5.6 C4: Sealed-Payload Confidentiality

Setup. Property 4 states that a sealed task payload decrypts only with the private key matching the peer’s appraised channel key. The peer generates a channel keypair; we seal a secret payload to its public key with `ca2a_runtime.channel` and check four sub-properties: (1) the sealed blob does not contain the plaintext; (2) the peer’s private key recovers it; (3) an unrelated key cannot open it; and (4) a tampered ciphertext fails closed under AEAD authentication.

Results. All four checks pass (4/4). A different key and a flipped byte both raise `SealedChannelError` rather than returning corrupted plaintext. This validates payload confidentiality at the cryptographic layer. It says nothing about whether the peer’s host can read the private key; that is the hardware dependency Property 4 names, and Section 6 reports platform state that bears on it.

Throughput and latency. We measured the sealed channel across payload sizes from a 64-byte task envelope to a 256 KiB document, timing `seal` (ephemeral key generation, ECDH, HKDF, AEAD encryption) and `open` (ECDH, HKDF, AEAD decryption) over 2000 iterations each after warm-up, with a round-trip check confirming every opened payload equalled the original. A 64-byte envelope seals and opens

in 0.17ms round trip, a cost dominated by the fixed X25519 and HKDF setup. As payloads grow the setup amortizes and the AEAD dominates: end-to-end throughput rises from 0.37MiB/s at 64 bytes to 212MiB/s at 256KiB (July: 0.21ms and 266MiB/s). Figure 4 plots the rerun. Raw measurements are in `experiments/results/sealed_channel_throughput.json`.

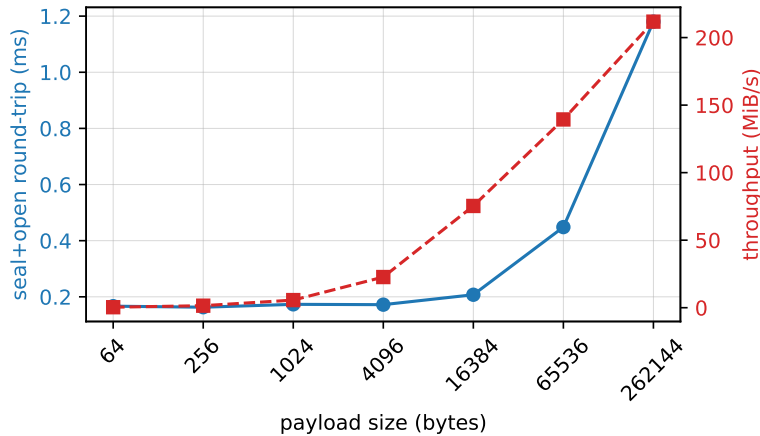


Figure 4: Sealed-channel round-trip latency (left axis) and end-to-end throughput (right axis) versus payload size, log-scale x-axis, 2000 timed `seal` and `open` iterations per size, ca2a 0.3.1. Small payloads are setup-dominated (0.17ms round trip at 64 bytes); large payloads are AEAD-dominated, reaching 212MiB/s at 256KiB.

5.7 C5: Provenance Integrity

Setup. From a signed three-hop chain the repository experiment emits one unsigned `DelegationRecord` per hop, each carrying the SHA-256 `record_hash` of its parent. It checks four sub-properties: (1) a valid record set verifies with `verify_dag`; (2) changing a record’s scope changes its `record_hash` and breaks its child’s parent link; (3) pointing a record at the wrong ancestor is detected; and (4) `cross_check_chain` binds record i to credential i .

Results. The valid record set verifies. Injecting one capability into a record’s scope changed 119 and 138 of the 256 bits of its `record_hash` in two runs, as expected of SHA-256 [Webster and Tavares, 1986], and `verify_dag` raised `ProvenanceLinkBroken` because the child’s stored parent hash no longer matched. Reparenting the leaf record was likewise detected, and `cross_check_chain` raised `ProvenanceLinkBroken` when a record’s `credential_id` was forged.

What this shows. These records are unsigned, so the experiment shows structural consistency against a single edit, not authenticity: a writer who rewrites every record can recompute every link and pass all four checks, and an edit to the leaf has no child link to break. Authenticity comes from the signed TRACE path (`verify_trace_dag` and `cross_check_trace_dag`), whose unit tests, including those binding each record’s signing key to the chain’s delegate, passed in the rerun. That path is not measured by a separate experiment in this report.

5.8 C6: Cross-Operator Attestation

Setup. The repository experiment builds two operators, each with its own channel keypair and its own synthetic VCEK key, and binds each channel key into a synthetic SEV-SNP report. Both synthetic VCEK certificates chain to one synthetic root standing in for the AMD ARK. Each side verifies the other’s report chain and signature with the shipped verifier, checks the measurement against an expected value, seals a delegated task to the key the report binds, and finally presents a validly signed report with a different measurement.

Results. All four checks pass (4/4): the keys are independent, each report’s channel key is recovered after verification, the sealed task opens only with the peer’s key, and the report with the swapped measurement is rejected with **AttestationFailed**. This exercises the protocol logic and the verifier code. It does not show that genuine hardware produces such reports for this protocol between independent operators. The recorded hardware runs (Section 3.5) were one-directional across two clouds or mutual within one operator, their raw evidence is not published, and they were not repeated for this edition. C6 therefore remains unvalidated on hardware.

6 Findings after the evaluation

The July 2026 draft was evaluated against an early checkout. Later work in the ca2a repository found gaps in that code and in the draft’s claims. They are listed here with the commit that addressed each and the first release containing it, so that the July results can be read against them. All are recorded in the project’s changelog.

A delegation chain was a bearer credential. The inbound path verified a chain and then granted its leaf authority without asking the caller to show any relationship to the leaf subject, so anyone holding a copy of a published chain could exercise it, and the provenance record would name the legitimate delegate. Property 2 as tested does not catch this, because nothing inside the chain changes. Since commit **cbfeb40** (0.2.0) the callee issues a challenge and requires a signature under the leaf subject’s key over the request, the channel key, and the payload digest.

Untrusted roots were accepted. A chain that was internally consistent verified whoever minted its root, so an attacker could grant itself any locally allowed capability. The runtime required a trusted root set from commit **e647e90** (0.2.0), and **verify_chain** itself from commit **23c980a** (0.3.0). The July C1 and C2 results were produced without a trust set; the rerun supplies one.

Unsigned provenance did not authenticate lineage. The July draft described the C5 records as verifiable “from signed records alone”, but the experiment used unsigned records whose hash links a writer can recompute. Since commit **550fba9** (0.3.0) the CLI separates authenticated lineage over signed TRACE records from structural diagnostics, and commit **e8b5cb7** (0.3.0) requires each signed record’s key to match the delegate named by the chain, which the cross-check had not compared.

Grants could not be revoked early. Until commit **f018c0e** (0.3.0) a credential stayed valid for its whole window. Issuers can now sign revocation statements that a verifier consults through a supplied snapshot. cA2A does not publish or fetch revocation data, and a verifier without a snapshot reports revocation as not checked.

The TDX parser rejected every genuine quote. Real DCAP v4 quotes nest the quoting-enclave data under a type-6 header that the parser and its synthetic fixtures both omitted, so the TDX path had never accepted real evidence. The failure was closed rather than an unsound accept. It was fixed in commit **82be440** (0.2.0) when a genuine quote was first appraised, which is why this edition re-appraised a public genuine quote rather than relying on synthetic vectors.

Measured hosts lacked platform protections. The GCP SEV-SNP hosts used on August 24 and September 17, 2026 reported **PLATFORM_INFO** 0x25: SMT enabled and ciphertext hiding not enabled. Stricter platform policies correctly refused them. cA2A can enforce such a policy, but appraisal of platform state is opt in and off by default. In the September 17 diagnostic the launch measurement also did not cover application code installed after boot. Both bear directly on Property 4: a hardware-appraised channel key on such a host is not shown to be out of the host operator’s reach.

Malformed input escaped as exceptions. Before commit **66ad9f0** (0.3.1) some malformed wire documents, TDX quotes, and MAC strings raised Python exceptions instead of the documented error, and the canonicalizer coerced non-string keys so that two distinct objects could serialize alike. Each now fails with a documented error.

7 Related Work

cA2A draws on three lines of work: capability-based delegation, identity and transparency standards, and confidential computing for agents. Several of its mechanisms appear in prior or concurrent work, and we do not claim priority for attenuation or for binding provenance to authority. What we did not find in the work below is the combination of per-hop attenuation with appraisal of the delegate’s TEE evidence and sealing of the task payload to the key that evidence vouches for, on the A2A delegation edge.

Capability delegation for agents. Macaroons [Birgisson et al., 2014] introduced contextual caveats that attenuate a bearer credential without a round trip to the issuer; a derived credential can only be narrower, and the check is offline. Tomašev, Franklin, and Osindero propose delegation capability tokens for AI agents based on macaroons or Biscuit tokens, wrapping resource credentials in attenuating caveats [Tomašev et al., 2026]. Prakash’s Agent Identity Protocol (AIP) implements invocation-bound capability tokens that combine identity, attenuated authorization, and provenance in an append-only token chain, with Biscuit and Datalog policies for multi-hop delegation, bindings for MCP, A2A, and HTTP, and interoperable Python and Rust implementations [Prakash, 2026]. The PEDIGREE Internet-Draft, an individual submission from Glyphzero Labs that extends the SPIFFE and WIMSE workload-identity model, specifies per-hop delegation with scope attenuation enforced at mint and at verification, Cedar-policy mandates with static-analysis proofs that a child mandate narrows its parent, and cryptographic re-verification of each parent token to resist parent-swap attacks [Rampalli, 2026b]. These cover the attenuation and chain-integrity ground of Properties 1 and 2, and PEDIGREE’s narrowing proofs over Cedar policies go further than cA2A’s set-subset check. Neither AIP nor PEDIGREE addresses appraisal of the delegate’s runtime attestation or encryption of the task payload to the delegate. cA2A’s delegation semantics mirror agent-manifest [Siddique, 2026].

Identity and transparency standards. The IETF WIMSE architecture draft [Salowey et al., 2026] describes how workloads present and verify identity across systems, and the SCITT architecture, published as RFC 9943 in June 2026 [Birkholz et al., 2026], defines transparency services for signed supply-chain statements. The same author as PEDIGREE has proposed binding per-action authorization and memory provenance into SCITT agent action capsules [Rampalli, 2026a]. cA2A’s per-hop records follow the SCITT pattern of signed statements that a verifier can appraise offline, and the TRACE record format it emits [Poddar et al., 2026b] has a transparency field that can carry a SCITT receipt; cA2A does not itself operate a transparency service. From the adversary side, Anbiaee et al. compare the security of MCP, A2A, Agora, and ANP and identify protocol-level and cross-protocol risks, including missing validation and attestation of executable components [Anbiaee et al., 2026].

Agent identity and messaging infrastructure. AGNTCY, initiated by Cisco and hosted by the Linux Foundation since July 2025, provides agent discovery, identity, and messaging components for multi-agent systems [AGNTCY Project, 2026]. It addresses naming, discovery, and authenticated communication at a scale cA2A does not attempt. We did not find in its published components a mechanism that intersects a delegated scope with the receiver’s local policy or seals a payload to an appraised TEE key. The two could compose: such a layer can name the peers while cA2A governs the authority that flows between them.

Hardware-attested provenance. EQTY Lab’s Verifiable Compute uses TEE attestation to notarize AI computation, pairing Intel TDX CPUs with NVIDIA H100 GPUs in its published design [EQTY Lab, 2024], and its Verifiable Knowledge work extends that approach to the provenance of data an agent uses, on NVIDIA BlueField hardware [EQTY Lab, 2026]. These are commercial systems built around particular hardware pairings. cA2A’s verifier follows the RATS role separation [Birkholz et al., 2023] and appraises SEV-SNP, TDX, and TPM evidence through one interface, but it has not been validated across independent operators (Property 6).

Confidential agentic AI. Forough, Kogias, and Haddadi survey confidential computing for agentic AI across six TEE platforms and list open challenges, among them compound attestation for multi-hop agent chains [Forough et al., 2026]; Mo, Tarkhani, and Haddadi systematize confidential computing for machine

learning more broadly [Mo et al., 2024]. cA2A’s sealed channel (Property 4) targets the delegation hop, and Section 6 records why it is not yet a confidentiality guarantee against a hosting operator. The sibling cMCP profile [Poddar et al., 2026a] applies confidential computing to the single-agent tool-call boundary; cA2A applies the same TEE provider pattern and Cedar engine to the agent-to-agent boundary.

8 Conclusion

A2A standardizes how agents discover each other and exchange tasks, but its trust model stops at the Signed Agent Card. Card verification establishes that a domain owner issued a card. It does not establish that the peer runs attested code, that a delegating agent holds the authority it passes on, that the task payload is confidential to the peer, or that there is a record of who delegated what to whom. cA2A is a trust profile layered on A2A that addresses these with attenuated delegation, attestation-gated sealing, and signed per-hop provenance, reusing existing standards rather than defining a new transport [A2A Project, 2026, Poddar et al., 2026b].

Rerun against ca2a 0.3.1, the software results hold. Attenuation was exact across 2,700 narrowing and 2,700 escalating chains; in-chain replay and cross-chain splices were rejected; delegated scope was intersected with local policy, including on the reference inbound path; sealed payloads opened only with the peer’s key and failed closed on tamper; and edits to linked provenance records were detected. Chain verification cost about 0.22 ms per hop in our environment, and the sealed channel reached 212 MiB/s at 256 KiB.

The limits are as important as the results. The provenance experiment shows structural consistency; authenticity depends on signed records, which the project’s tests cover. The sealed channel is validated at the cryptographic layer only: the measured hosts lacked SEV-SNP ciphertext hiding, and the launch measurement in the recorded mutual run did not cover application code. Cross-operator attestation has been exercised with synthetic evidence and, on hardware, only one-directionally or within one operator. Section 6 lists defects found after the original evaluation, including chains that acted as bearer credentials and unsigned provenance presented as authenticated; each was fixed in a released version.

Until mutual attestation between independent operators is demonstrated on platforms that provide the needed memory protections, cA2A should not be described as attested across trust domains or as protecting payloads from a hosting operator. The specification, verifier, and experiments are available at <https://github.com/agenttrust-io/ca2a> under the MIT license.

References

- A2A Project. Agent2Agent (A2A) protocol specification, version 1.0. <https://a2a-protocol.org/latest/specification/>, 2026. Accessed September 2026.
- Martín Abadi and Cédric Fournet. Access control based on execution history. In Proceedings of the 10th Network and Distributed System Security Symposium (NDSS), 2003. URL <https://www.ndss-symposium.org/ndss2003/access-control-based-execution-history/>.
- Advanced Micro Devices, Inc. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. White paper, AMD, January 2020. URL <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>.
- Agentic AI Foundation. A2A joins AAIF’s open agentic stack. <https://aaif.io/blog/a2a-joins-aaif>, August 2026. Published August 17, 2026. Accessed September 2026.
- AGNTCY Project. AGNTCY: Building the internet of agents. <https://agntcy.org/>, 2026. Linux Foundation project since July 2025. Accessed September 2026.
- Zeynab Anbiaee, Mahdi Rabbani, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Ali Ghorbani, and Sajjad Dadkhah. Security threat modeling for emerging AI-agent protocols: A comparative analysis of MCP, A2A, Agora, and ANP. arXiv:2602.11327, 2026. URL <https://arxiv.org/abs/2602.11327>.

- Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In Proceedings of the Network and Distributed System Security Symposium (NDSS), 2014. URL <https://www.ndss-symposium.org/ndss2014/macaroons-cookies-contextual-caveats-decentralized-authorization-cloud>.
- Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, RFC Editor, January 2023. URL <https://www.rfc-editor.org/rfc/rfc9334>.
- Henk Birkholz, Antoine Delignat-Lavaud, Cédric Fournet, Yogesh Deshpande, and Steve Lasker. An architecture for trustworthy and transparent digital supply chains. RFC 9943, RFC Editor, June 2026. URL <https://www.rfc-editor.org/rfc/rfc9943>.
- cA2A Contributors. ca2a-runtime 0.3.1: Confidential agent-to-agent runtime. <https://github.com/agenttrust-io/ca2a>, 2026. Tag v0.3.1, commit 7d90823142dda1be589def3b22a3099581c7df7a, September 26, 2026.
- Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. Proceedings of the ACM on Programming Languages, 8(OOPSLA1):670 to 697, 2024. doi: 10.1145/3649835.
- EQTY Lab. Verifiable compute. White paper, version 1.08, EQTY Lab, December 2024. URL <https://a.storyblok.com/f/257174/x/7b61f19b05/verifiable-compute-white-paper-december-2024.pdf>.
- EQTY Lab. Verifiable knowledge: The third pillar of trust for agentic AI. <https://www.eqtylab.io/blog/verifiable-knowledge-the-third-pillar-of-trust-for-the-agentic-ai>, June 2026. Published June 1, 2026. Accessed September 2026.
- Javad Forough, Marios Kogias, and Hamed Haddadi. When agents handle secrets: A survey of confidential computing for agentic AI. arXiv:2605.03213, 2026. URL <https://arxiv.org/abs/2605.03213>.
- Nicola Gallo. Proof-of-continuity: A temporal model for authority propagation in distributed systems and AI agents. arXiv:2607.08906, 2026. URL <https://arxiv.org/abs/2607.08906>. Model and drafts at <https://github.com/ngallo/pic-model>.
- Norm Hardy. The confused deputy (or why capabilities might have been invented). ACM SIGOPS Operating Systems Review, 22(4):36 to 38, 1988. doi: 10.1145/54289.871709.
- Intel Corporation. Intel TDX module base architecture specification. Technical report, Intel Corporation, May 2026. URL <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html>.
- Simon Josefsson and Ilari Liusvaara. Edwards-Curve digital signature algorithm (EdDSA). RFC 8032, RFC Editor, January 2017. URL <https://www.rfc-editor.org/rfc/rfc8032>.
- Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, RFC Editor, June 2013. URL <https://www.rfc-editor.org/rfc/rfc6962>.
- Laurence Lundblade, Giridhar Mandyam, Jeremy O'Donoghue, and Carl Wallace. The entity attestation token (EAT). RFC 9711, RFC Editor, April 2025. URL <https://www.rfc-editor.org/rfc/rfc9711>.
- Fan Mo, Zahra Tarkhani, and Hamed Haddadi. Machine learning with confidential computing: A systematization of knowledge. ACM Computing Surveys, 56(11):1 to 40, 2024. doi: 10.1145/3670007.
- Model Context Protocol Project. Model context protocol specification, revision 2026-07-28. <https://modelcontextprotocol.io/specification/2026-07-28>, 2026. Accessed September 2026.

- Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3C Recommendation, April 2013. URL <https://www.w3.org/TR/prov-dm/>.
- Jaehong Park, Dang Nguyen, and Ravi Sandhu. A provenance-based access control model. In Tenth Annual International Conference on Privacy, Security and Trust (PST), 2012.
- Rishabh Poddar, Aaron Fulkerson, and Imran Siddique. cMCP: Verifiable policy enforcement for AI agent tool calls. Technical report, version 1, OPAQUE Systems, September 2026a. URL <https://doi.org/10.5281/zenodo.23001694>.
- Rishabh Poddar, Aaron Fulkerson, and Imran Siddique. TRACE: Trust, runtime attestation, and compliance evidence (A portable attestation format for AI agent runtime governance). Technical report, version 1, OPAQUE Systems, September 2026b. URL <https://doi.org/10.5281/zenodo.23001692>.
- Sunil Prakash. AIP: Agent identity protocol for verifiable delegation across MCP and A2A. arXiv:2603.24775, 2026. URL <https://arxiv.org/abs/2603.24775>.
- Karthik Rampalli. Binding per-action authorization and memory provenance into agent action capsules. Internet-Draft draft-rampalli-scitt-capsule-provenance-binding-00, individual submission, July 2026a. URL <https://datatracker.ietf.org/doc/draft-rampalli-scitt-capsule-provenance-binding/>. Work in progress.
- Karthik Rampalli. PEDIGREE: Verifiable delegation identity for agentic AI systems. Internet-Draft draft-rampalli-pedigree-00, individual submission, April 2026b. URL <https://datatracker.ietf.org/doc/draft-rampalli-pedigree/>. Work in progress.
- Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). RFC 8785, RFC Editor, June 2020. URL <https://www.rfc-editor.org/rfc/rfc8785>.
- Joseph A. Salowey, Yaroslav Rosomakho, and Hannes Tschofenig. Workload identity in a multi system environment (WIMSE) architecture. Internet-Draft draft-ietf-wimse-arch-08, July 2026. URL <https://datatracker.ietf.org/doc/draft-ietf-wimse-arch/>. Work in progress.
- Imran Siddique. Agent manifest: Portable, hardware-attestable identity and provenance for AI agents. Technical report, version 1, OPAQUE Systems, September 2026. URL <https://doi.org/10.5281/zenodo.23001698>.
- Nenad Tomašev, Matija Franklin, and Simon Osindero. Intelligent AI delegation. arXiv:2602.11865, 2026. URL <https://arxiv.org/abs/2602.11865>.
- A. F. Webster and S. E. Tavares. On the design of S-boxes. In Advances in Cryptology: CRYPTO '85 Proceedings, volume 218 of Lecture Notes in Computer Science. Springer, 1986. doi: 10.1007/3-540-39799-X_41.